```
int a=4;
int b=6;
int c;
c= a+b;
Printf(%d,c);
}
```

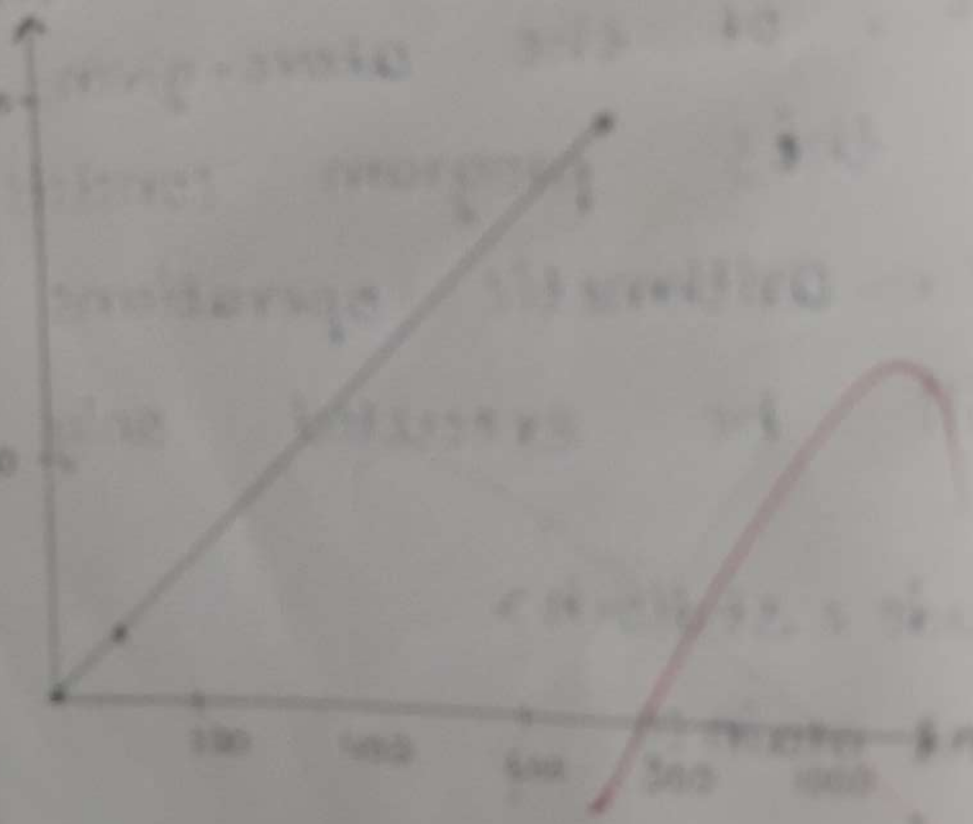
Ex 2

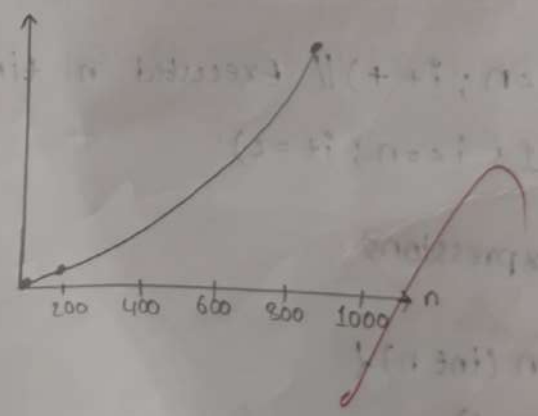
Linear time - O(n)

An algorithm is said to have a linear time complexity when the running time increases linearly with the length of the inpu when the function involves checking all the values in input data, with this order o

Time complexity calculation: Loop variables is incremented / decremented by a constant amoun Even though the inner loop is bounded by n, but due to break statement is executing only once.

```
for (int i=0; i<n; i++) // Executed n times
  for (int i=1; i<=n; i+=c)
   {
   // some expressions
   }
    function (int n) {
      if (n==1)
        return;
```


500
100
150
500
500
500
1000


200
400
600
800
1000
n

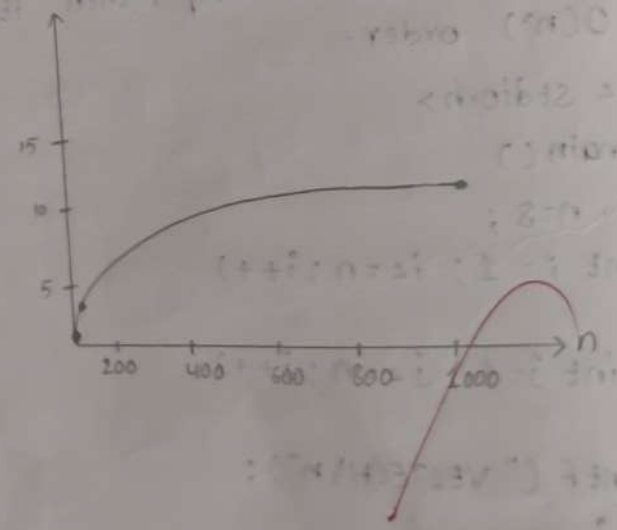
```
for (int i=1; i<=n; i++)
 {
    for (int i=1; j<=n; j++)
    {
       Printf ("*")
        break;
        } }
          int m=0;
          for (int i=0; i<n; i++) {
                m=m+1;
                }
```

Ex 3

Quadratic time - $O(n^2)$

An algorithm generally, nested loops come under this order where one loop takes $O(n)$ and if the function involves a loop within a loop, then it goes for $O(n)*O(n) = O(n^2)$ order.

```
# include < stdio.h>
      int main()
     int i, j, n=8;
     for (int i=1; i<=n; i++)
        {
       for( int j=1; j<=n; j++)
        {
         PrintF (" VELTECH\n");
         }}}
     for (int i=1; i<=n; i+=c)
       {
        for (int j=1; j<=n; j+=c)
        {
            //some expressions
              }
```


15
10
5
200
400
600
800
1000
n

```
}
int m=0
    // Executed n times
    for (int i=0 ; i<n; i++) {
        m=m+1;
    }
    // outerloop executed n times        n+n*n -> O(n²)
    for (int j=0; j<n ; j++)
        m= m+1 ;
    }
```

Ex 4

Logarithmic time - O (log n)

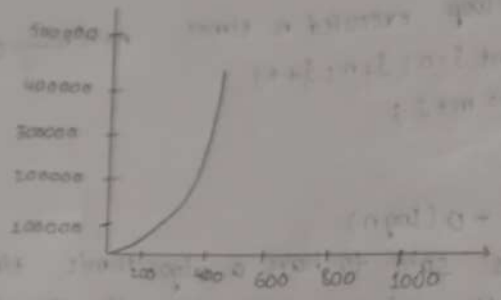
An algorithm is said to have a algorithmic time complexity when it reduces the size of the input data in each step. This involves the search of a given value in an array by splitting the array into two and starting searching in one split.

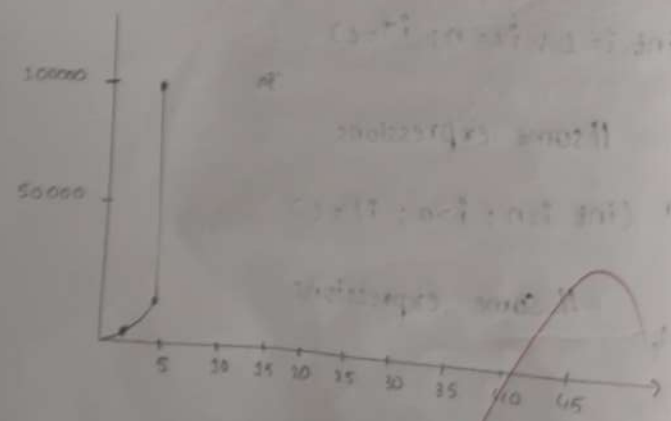
```
for (int i= 1 ; i<= n; i*=c)
{
        // some expressions
}
for (int i=n ; i>0 ; i/=c)
{
        // Some expressions
}
```

Ex 5

O (n log n) — A log-linear Algorithm

O(n log n) is known as log linear complexity


500000
400000
300000
200000
100000
100
400
600
800
1000


100000
50000
5
10
15
20
25
30
35
40
45

O(n log n) implies that log n operations will occur n times. O(n log n) times is common in recursive sorting algorithms, sorting algorithms using a binary tree sort, and most other types of sorts

MERGE SORT

Time complexity calculation:

Merge sort is a recursive algorithm and time complexity can be expressed as $T(n) = 2T(n/2) + O(n)$

$O(n^* \log n)$ worst case time complexity : $O(n^* \log n)$

Average time complexity : $O(n^* \log n)$ in all 3 cases as the merge sort always divides the array into two halves and takes linear time to merge two halves.

Ex 5

Exponential Time

Exponential growth is the opposite of logarithmic growth becomes efficient after a certain number of inputs but exponential gets slower and inefficient as the the input size grows.

Ex 6

Cubic time - $O(n^3)$

An algorithm is said to run in cubic time if the running time of the three loops is

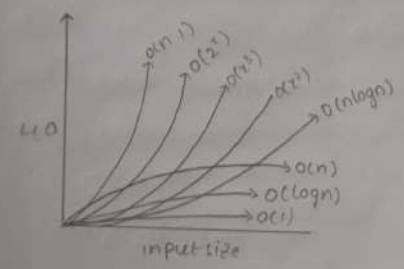
Proportional to the cube of N. where N triples, the running time increases by N*N*N.

```
int m=0
// outer loop executed n/2 times
    for (int i=n/2; i<n; i++){
// middle loop executed n/2 times
    for (int j=n/2; j<n; j++)
// inner loop executed n times
    for (int k=0; k<n; k++)
        m=m+1;
}
```

Time complexity calculation:

```
//outer loop executed n/2 times
    for(int i=n/2; i<n; i++)
//middle loop executed n/2 times
    for(int j=n/2; j<n; j++)
//inner loop executed n times
    for(int k=0; k<n; k++)
```

complexity will be $n/2 * n/2 * n = n^3$



VEL TECH - CSE	
EX NO.	1
PERFORMANCE (5)	5
RESULT AND ANALYSIS (5)	5
VIVA VOCE (5)	5
RECORD (5)	5
TOTAL (20)	20
SIGNATURE	

10/2/26

Result: Thus calculating the time complexities for the given coding and estimating the orders of growth was learn sucessfully.